Section C

Development Environment and Technical Preparation

Universe 25 Population Simulator, APPDAET Final Project, Group 2

DRAFT

C.1 Install and Configure Python and the Selected IDE

We use PyCharm as our IDE, with Python 3.14.5 on Windows. The project is called Universe25_Simulator and uses a virtual environment (.venv) created by PyCharm. We tested the setup by importing tkinter and matplotlib, and it worked.

C.2 Required Python Libraries and Dependencies

The project needs only one outside library, matplotlib, which we installed with pip install matplotlib (version 3.11.2). Everything else is built into Python. Matplotlib only draws the charts, and the births, deaths, and resources are computed by our own code.

Library	**Purpose**
tkinter	Window, input fields, buttons, and the table
random	Random births and deaths, with an optional seed
matplotlib	Population and resource charts
unittest	Testing

C.3 Datasets, Formulas, and Supporting Resources

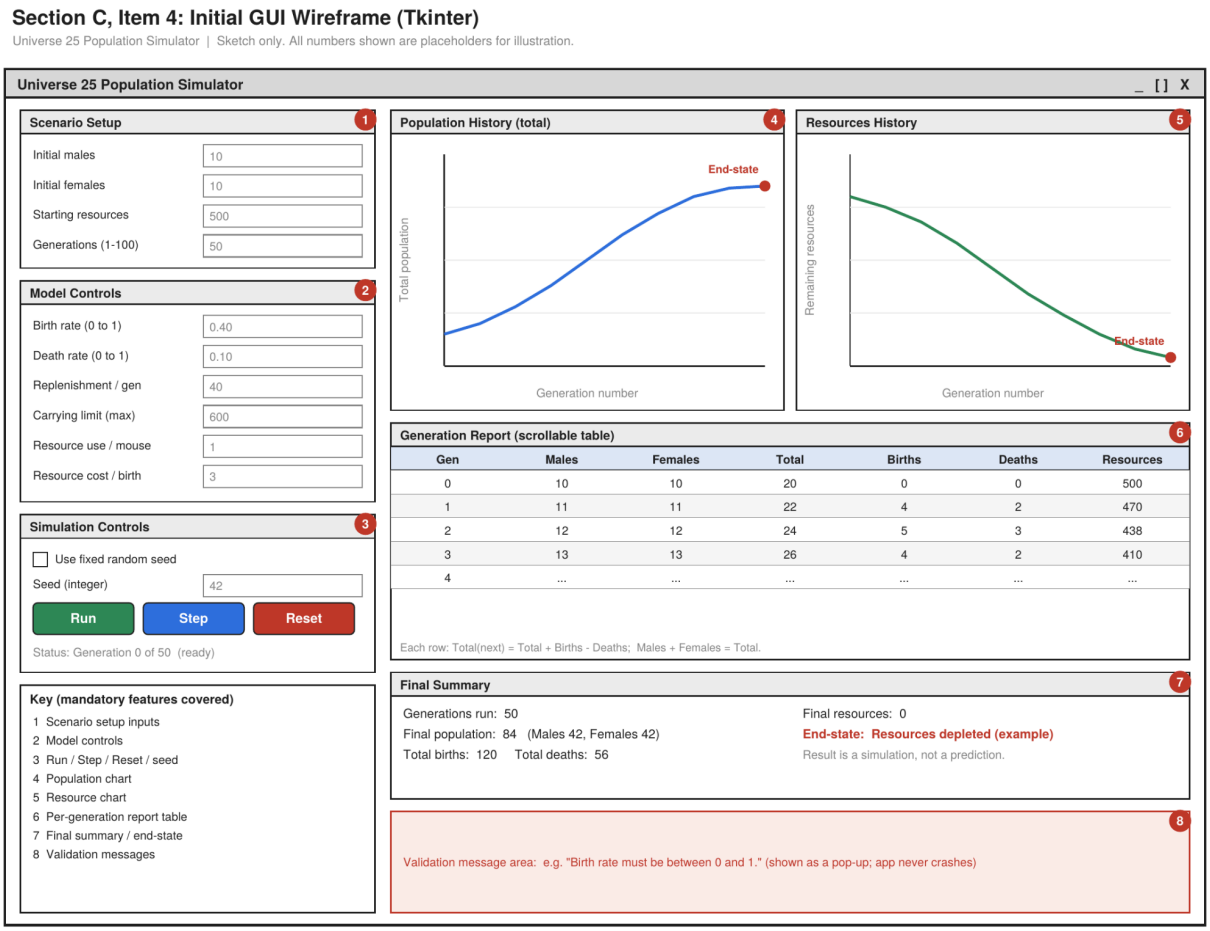
No dataset is needed because the simulation uses the numbers the user enters. The model is a simple random model, and using the same seed gives the same result. These formulas are proposals, and Part B may change them.

Each generation, the number of pairs is the smaller of males and females. Each pair has a chance (the birth rate) of having a baby, but births are limited by the resources available. Each mouse has a chance (the death rate) of dying. Resources go down as mice use them and go up by the replenishment amount, and they stay between 0 and the limit. The new population is the old population plus births minus deaths, and it never goes below 0.

This also covers the required tests: no females means no births, no mice means the population stays at 0, and not enough resources means no births.

C.4 Initial GUI Sketch (Wireframe)

The left side has the inputs: scenario setup, model controls, and the Run, Step, Reset, and seed controls. The right side shows the population chart, the resource chart, the generation table, and the final summary. Invalid inputs show an error message instead of crashing.



C.5 Proposed Organization of the Application

We split the program into a few files so the math, the program state, and the window stay separate.

File	Responsibility
main.py	Starts the app
model.py	The algorithm (births, deaths, resources)
validation.py	Checks the inputs
state.py	Keeps the current run and handles Step, Run, and Reset
gui.py	Window, table, and charts
tests/	Automated tests

When the user clicks Step, gui.py checks the inputs, state.py asks model.py to compute the next generation, and gui.py updates the table and charts. Run repeats this until the last generation, and Reset goes back to generation 0.